

Laboratório 10 — Contador de 4 Bits em VHDL

Pontifícia Universidade Católica de Campinas — Engenharia de Computação
João Paulo Nunes Andrade - 25002703

1. Objetivo

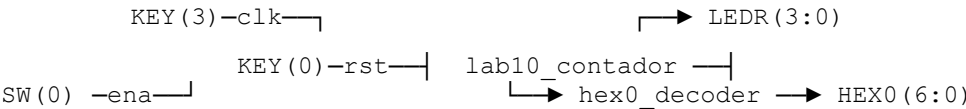
O laboratório consiste em implementar, em VHDL, um contador crescente síncrono de 4 bits em uma FPGA (placa DE-série da Altera/Intel). O botão KEY(3) atua como sinal de clock, SW(0) como enable, e KEY(0) como reset assíncrono. O valor do contador é exibido simultaneamente nos LEDs vermelhos LEDR(3..0) em binário e no display HEX0 em hexadecimal.

2. Arquitetura da Solução

A solução foi dividida em dois módulos VHDL distintos: um componente combinacional responsável pela decodificação do valor binário para o display de 7 segmentos (`hex0_decoder`), e um módulo top-level (`lab10_contador`) que instancia o decodificador e implementa a lógica sequencial do contador.

2.1 Diagrama de Blocos

O fluxo de dados parte do contador interno do módulo top-level. O valor de 4 bits alimenta simultaneamente os 4 LEDs vermelhos e a entrada do decodificador, cuja saída de 7 bits controla os segmentos do HEX0.



2.2 Tabela de Portas — Top-Level

Sinal	Direção	Largura	Descrição
KEY(3)	in	1 bit	Clock — borda de descida conta
KEY(0)	in	1 bit	Reset assíncrono ativo em '0'
SW(0)	in	1 bit	Enable — '1' habilita contagem
LEDR(3:0)	out	4 bits	Valor binário do contador
HEX0(6:0)	out	7 bits	Valor em hexadecimal (7 seg.)

3. Lógica Sequencial do Contador

O contador utiliza um flip-flop de 4 bits com reset assíncrono. A sensibilidade do processo é declarada em (`clk`, `resetn`), de modo que o reset atua imediatamente quando KEY(0) vai a nível baixo, independente do estado do clock. A contagem ocorre na borda de descida de KEY(3), que é o comportamento físico dos botões da placa (ativos em '0').

3.1 Tabela de Transição de Estados

Estado Atual	resetrn / enable	Próximo Estado	Condição
Q (qualquer)	'0' / X	0000	Reset assíncrono
Q	'1' / '0'	Q	Hold (enable desativado)
Q (0000..1110)	'1' / '1'	Q + 1	Incremento
1111	'1' / '1'	0000	Overflow (volta a zero)

O overflow de 15 para 0 é tratado naturalmente pela aritmética de `std_logic_unsigned`: ao somar 1 a "1111", o resultado de 5 bits seria "10000", mas com 4 bits apenas os quatro bits menos significativos são mantidos, resultando em "0000".

4. Decodificador 7 Segmentos (hex0_decoder)

O display HEX0 da placa é do tipo ânodo comum — seus segmentos acendem com nível lógico '0'. O módulo `hex0_decoder` implementa um processo combinacional com `case` que mapeia cada um dos 16 estados possíveis (0x0 a 0xF) para o vetor de 7 bits correspondente, na ordem g-f-e-d-c-b-a.

4.1 Codificação dos Segmentos

A figura abaixo mostra a numeração convencional dos segmentos:

```

      |_ |      ← a (bit 0)
      |_ |      ← f (bit 5), g (bit 6), b (bit 1)
      |_ |      ← e (bit 4), d (bit 3), c (bit 2)

```

4.2 Tabela Verdade — hex0_decoder

bin_in	g	f	e	d	c	b	a	Char
0000	1	0	0	0	0	0	0	0
0001	1	1	1	1	0	0	1	1
0010	0	1	0	0	1	0	0	2
0011	0	1	1	0	0	0	0	3
0100	0	0	1	1	0	0	1	4
0101	0	0	1	0	0	1	0	5
0110	0	0	0	0	0	1	0	6
0111	1	1	1	1	0	0	0	7
1000	0	0	0	0	0	0	0	8
1001	0	0	1	0	0	0	0	9
1010	0	0	0	1	0	0	0	A
1011	0	0	0	0	0	1	1	b
1100	1	0	0	0	1	1	0	C

1101	0	1	0	0	0	0	1	d
1110	0	0	0	0	1	1	0	E
1111	0	0	0	1	1	1	0	F

5. Implementação VHDL

5.1 hex0_decoder.vhd

```

library ieee;
use ieee.std_logic_1164.all;

entity hex0_decoder is
    port (
        bin_in  : in  std_logic_vector(3 downto 0);
        hex_out  : out std_logic_vector(6 downto 0)
    );
end entity hex0_decoder;

architecture combinacional of hex0_decoder is
begin
    process(bin_in)
    begin
        case bin_in is
            when "0000" => hex_out <= "1000000"; -- 0
            when "0001" => hex_out <= "1111001"; -- 1
            when "0010" => hex_out <= "0100100"; -- 2
            when "0011" => hex_out <= "0110000"; -- 3
            when "0100" => hex_out <= "0011001"; -- 4
            when "0101" => hex_out <= "0010010"; -- 5
            when "0110" => hex_out <= "0000010"; -- 6
            when "0111" => hex_out <= "1111000"; -- 7
            when "1000" => hex_out <= "0000000"; -- 8
            when "1001" => hex_out <= "0010000"; -- 9
            when "1010" => hex_out <= "0001000"; -- A
            when "1011" => hex_out <= "0000011"; -- b
            when "1100" => hex_out <= "1000110"; -- C
            when "1101" => hex_out <= "0100001"; -- d
            when "1110" => hex_out <= "0000110"; -- E
            when "1111" => hex_out <= "0001110"; -- F
            when others => hex_out <= "1111111"; -- apagado
        end case;
    end process;
end architecture combinacional;

```

5.2 lab10_contador.vhd (Top-Level)

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

entity lab10_contador is
    port (
        KEY   : in  std_logic_vector(3 downto 0);
        SW    : in  std_logic_vector(0 downto 0);
        LEDR  : out std_logic_vector(3 downto 0);
        HEX0  : out std_logic_vector(6 downto 0)
    );
end entity lab10_contador;

architecture estrutural of lab10_contador is
    component hex0_decoder is
        port (
            bin_in  : in  std_logic_vector(3 downto 0);
            hex_out  : out std_logic_vector(6 downto 0)
        );
    end component;

```

```

    );
end component;

signal contador : std_logic_vector(3 downto 0) := "0000";
alias clk      : std_logic is KEY(3);
alias resetn   : std_logic is KEY(0);
alias enable   : std_logic is SW(0);
begin
    process(clk, resetn)
    begin
        if resetn = '0' then
            contador <= "0000";
        elsif falling_edge(clk) then
            if enable = '1' then
                contador <= contador + 1;
            end if;
        end if;
    end process;

    LEDR <= contador;

    U1 : hex0_decoder
        port map (
            bin_in  => contador,
            hex_out => HEX0
        );
end architecture estrutural;

```

6. Observações de Síntese

Ao criar o projeto no Quartus Prime, os dois arquivos .vhd devem ser adicionados, com `lab10_contador` definido como entidade top-level. O mapeamento de pinos deve seguir o pin planner da placa utilizada (

DE2-115), associando KEY(3), KEY(0), SW(0), LEDR e HEX0 aos pinos físicos corretos.

Um ponto de atenção é o comportamento de KEY: na placa, os botões não possuem debounce por hardware, o que pode gerar múltiplos pulsos por pressionamento em situações de contato instável. Para este laboratório, o comportamento é aceitável, mas em sistemas críticos seria necessário um circuito de debounce.

A biblioteca `std_logic_unsigned` foi escolhida por ser a abordagem típica em cursos de circuitos digitais. Para projetos mais modernos, a recomendação seria usar `ieee.numeric_std` com cast explícito para `unsigned`, porém ambas produzem síntese equivalente.